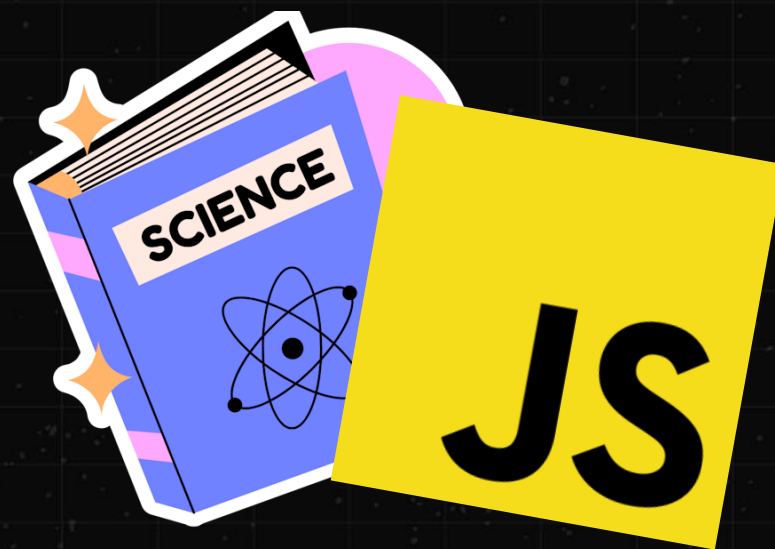


ЛЕКЦИЯ #10

CS BO FRONTEND



ЧТО СЕГОДНЯ НА ПОВЕСТКЕ

- * Будем говорить про **абстрактные структуры данных** на примере стека и очереди
- * Разберем различные варианты реализации
- * Познакомимся с новой фундаментальной структурой данных – **связным списком**

ЧТО ЕЩЕ ЗА АБСТРАКТНЫЕ СТРУКТУРЫ ДААННЫХ?



НИЧЕГО СЛОЖНОГО

- * Такие структуры данных говорят, **что нужно сделать: какой интерфейс и инварианты**
- * Но **не говорят как это сделать:** реализации могут отличаться
- * Например, спецификация JS говорит, что массивы – это **последовательность элементов с индексами, где могут быть пропуски**
- * Но не говорит, какая структура данных стоит за этим – в итоге **реализация может отличаться для каждой VM и меняться от версии к версии**

КАКОЙ СМЫСЛ В АБСТРАКТНОСТИ

- * Такие структуры **выделяют только главные контракты** – что позволяет легко менять реализацию по требованию
- * Использовать реализацию **лучше подходящую под конкретный случай**
- * Производить **рефакторинг и оптимизации**

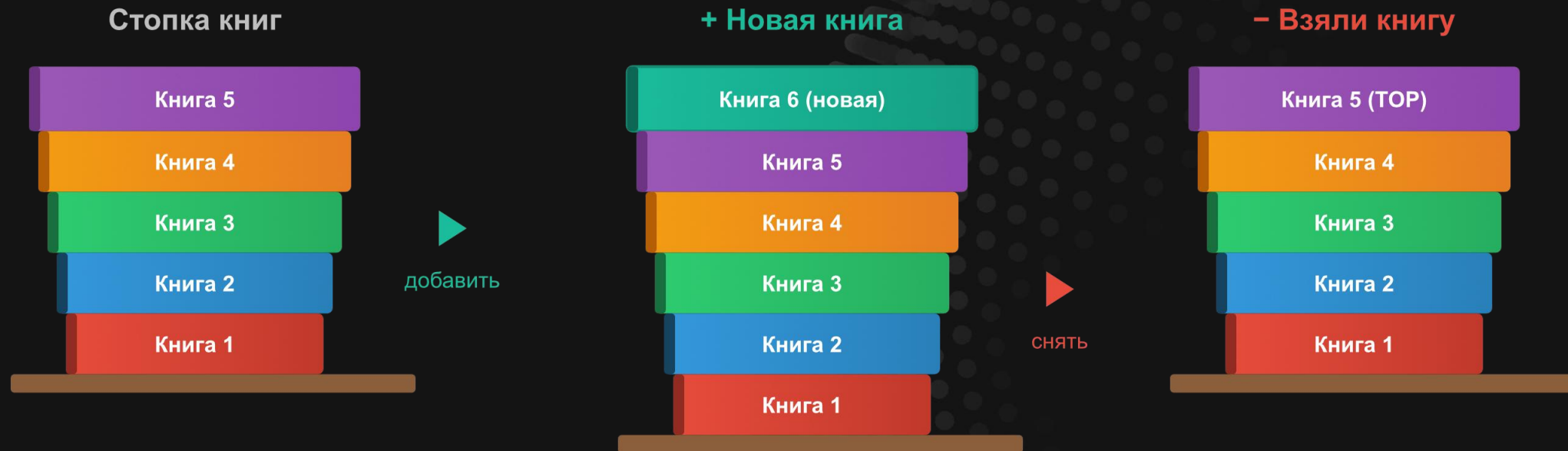
МОЖНО
КОНКРЕТНЫЕ
ПРИМЕРЫ?



СТЕК И ОЧЕРЕДЬ

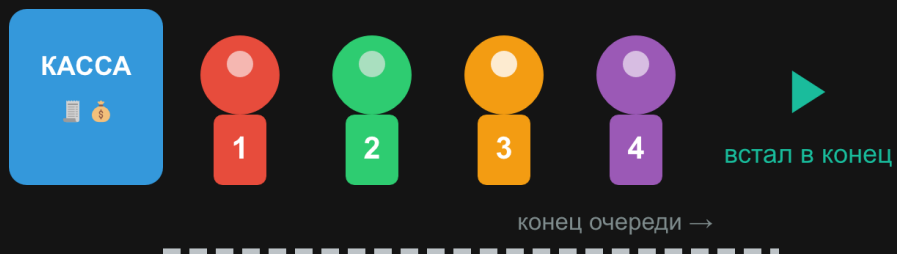
- * Две важнейшие абстрактные структуры данных
- * Обе описывают **упорядоченную последовательность элементов**
- * Обе разрешают **добавление новых элементов в конец последовательности**
- * Стек позволяет прочитать или удалить **последний элемент последовательности**
- * Очередь позволяет прочитать или удалить **первый элемент последовательности**
- * В идеале, эти операции должны иметь **среднюю амортизированную сложность $O(1)$**

СТЕК (LIFO: LAST IN – FIRST OUT)

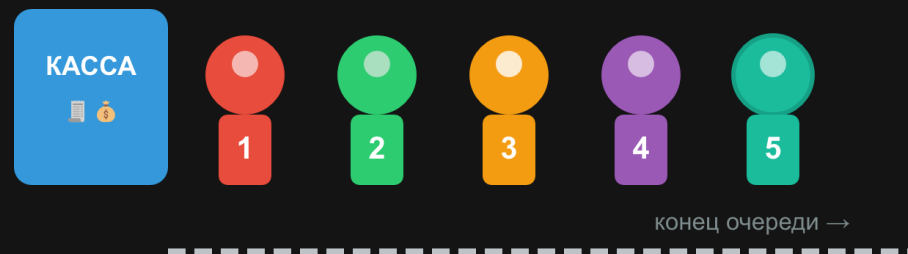


ОЧЕРЕДЬ (FIFO: FIRST IN – FIRST OUT)

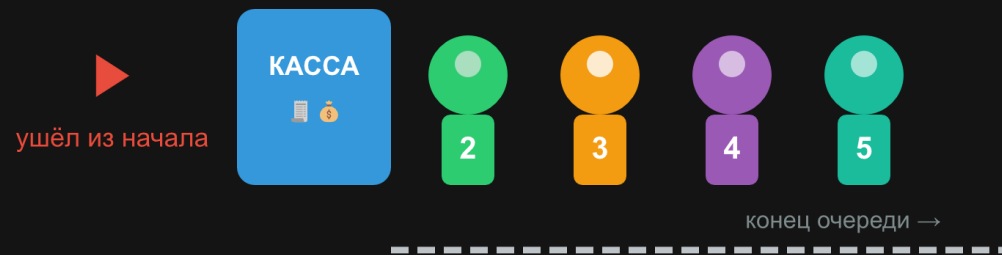
Очередь к кассе



+ Пришёл новый человек



– Человек обслужен



ЗАЧЕМ
НУЖНЫ ЭТИ
СТРУКТУРЫ?



ОНИ КРАЙНЕ ВАЖНЫ

- * Стек и очередь – это основа огромного количества алгоритмов
- * Стек используется при реализации функций в языках программирования
- * Наконец, необходимость в очередях возникает даже в простых прикладных задачах

ДВУСТОРОННЯЯ ОЧЕРЕДЬ

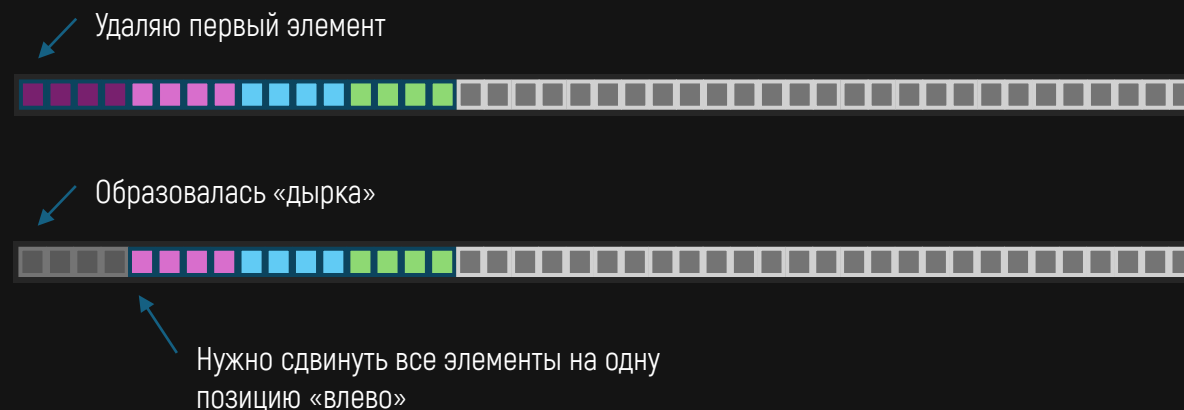
- * Также известна под названием «дек» (Deque – Double Ended Queue)
- * Такая структура данных, которая позволяет **добавлять и извлекать элементы с обоих концов последовательности**
- * Обычные массивы в JS реализуют как стек и очередь, так и дек (push/unshift/pop/shift)

ЭТИ СТРУКТУРЫ
РЕАЛИЗУЮТСЯ
ЧЕРЕЗ МАССИВ?



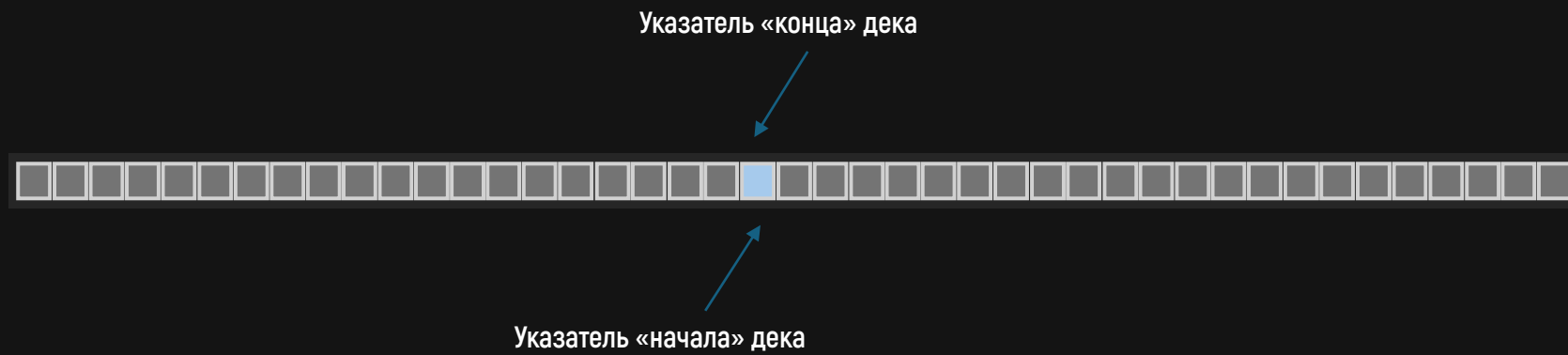
КАК ВАРИАНТ

- * **Стек на основе массива** очень прост и эффективен, а при использовании вектора появляется возможность динамического расширения
- * А вот **очередь или дек на массиве (как в JS)** уже не так хороши – дело в сложности $O(n)$ для добавления или удаления в начало массива

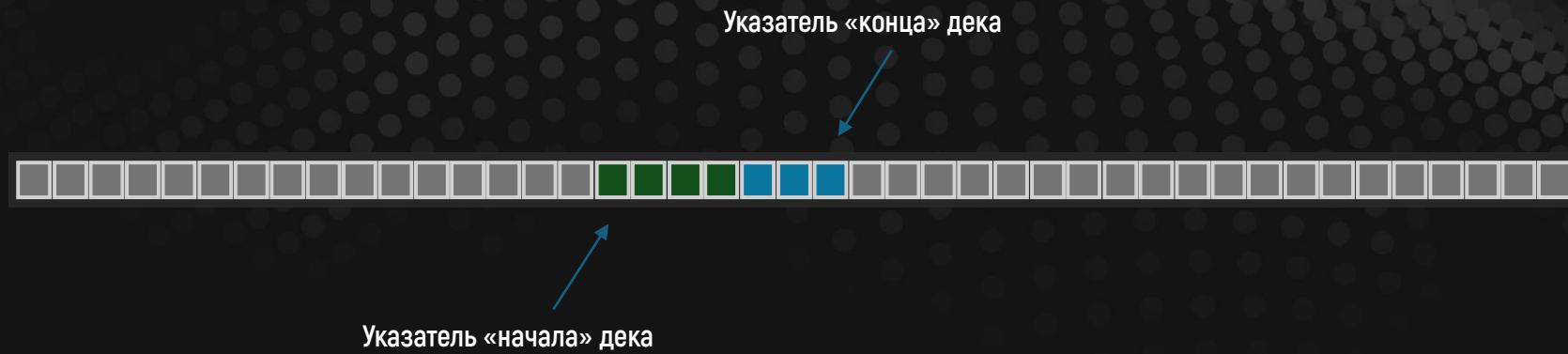


ДОБАВЛЕНИЕ В СЕРЕДИНУ

- * Обычно при добавлении элементов в массив – они добавляются с самого начала и следуют друг за другом до конца
- * Но можно начать добавлять элементы с середины массива – таким образом есть свободное место с обеих сторон массива
- * И держать два указателя на начало и конец последовательности
- * В итоге получаем $O(1)$ для всех 4-х операций (push/unshift/pop/shift)
- * Для динамического роста используем реаллокацию памяти



СДЕЛАЛИ 4 РАЗА UNSHIFT И 3 РАЗА PUSH



```

class Deque {
  get capacity() { return this.#buffer.length; }
  get length() { return this.#length; }

  #buffer; #start; #end;
  #length = 0;

  constructor(capacity = 4) {
    const actualCapacity = Math.max(4, capacity >>> 0);
    this.#buffer = new Array(actualCapacity);
    this.#start = Math.floor(actualCapacity / 2);
    this.#end = this.#start;
  }

  unshift(value) {
    if (this.#start <= 0) {
      this.resize(this.capacity * 2);
    }

    this.#start--;
    this.#buffer[this.#start] = value;
    this.#length++;

    return this.length;
  }

  push(value) {
    if (this.#end >= this.capacity) {
      this.resize(this.capacity * 2);
    }

    this.#buffer[this.#end] = value;
    this.#end++;
    this.#length++;

    return this.length;
  }
}

```

```

shift() {
  if (this.#length === 0) { return undefined; }

  const value = this.#buffer[this.#start];
  this.#buffer[this.#start] = undefined;
  this.#start++;
  this.#length--;

  return value;
}

pop() {
  if (this.#length === 0) { return undefined; }

  this.#end--;
  const value = this.#buffer[this.#end];
  this.#buffer[this.#end] = undefined;
  this.#length--;

  return value;
}

resize(newCapacity = this.#length) {
  if (newCapacity < this.#length) {
    newCapacity = this.#length;
  }

  const newBuffer = new Array(newCapacity);
  const offset = Math.floor((newCapacity - this.#length) / 2);

  // Копируем непрерывный блок в середину нового буфера
  for (let i = 0; i < this.#length; i++) {
    newBuffer[offset + i] = this.#buffer[this.#start + i];
  }

  this.#buffer = newBuffer;
  this.#start = offset;
  this.#end = offset + this.#length;
}

```


second monitor

\$0.00 (0%)

50.00

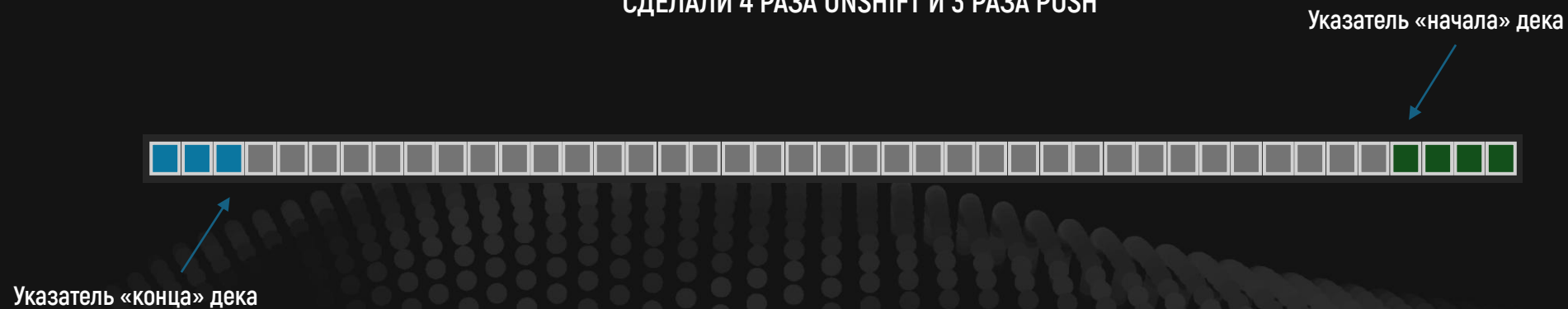
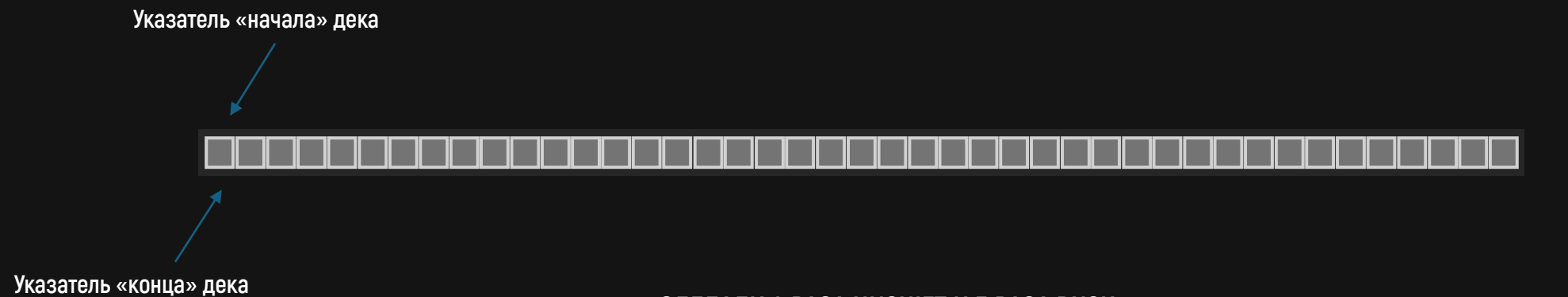
224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ

- * Мы уже разбирали этот код ранее на одном из созвонов
- * Мы избавились от необходимости «сдвигать» данные и **получили константную сложность операций дека**
- * Элементы дека лежат в памяти рядом, **что хорошо для кэширования**
- * Но если элементы добавляются только в начало или только в конец, то **память расходуется не эффективно (только половина)**
- * Решить эту проблему можно через **добавление элементов «с разных концов»**



УКАЗАТЕЛИ «ДВИГАЮТСЯ ПО КРУГУ»

- * Делая `push` мы двигаемся от 0 к концу
- * Делая `unshift` мы двигаемся от конца к 0
- * Операции `pop` и `shift` двигаются в обратном направлении от своего указателя
- * На самом деле, выбор «где начало и конец» для нас не важен
- * Куда важнее сама логика движения «по кругу»
- * Структура реализующая такой подход называется «кольцевым буфером»

А КАК РЕАЛИЗОВАТЬ «ДВИЖЕНИЕ ПО КРУГУ»?



ИСПОЛЬЗУЕМ ДЕЛЕНИЕ С ОСТАТКОМ

- * Чтобы «закольцевать» число внутри другого – нужно взять остаток от деления
- * Например, `this.#end = (this.#end + 1) % this.capacity;`
- * Когда `this.#end + 1` станет равен `this.capacity`, то операция `%` вернет `0`
- * Это напоминает целочисленное переполнение

```

class Deque {
  get capacity() { return this.#buffer.length; }
  get length() { return this.#length; }

  #buffer; #start; #end;
  #length = 0;

  constructor(capacity = 4) {
    const actualCapacity = Math.max(4, capacity >>> 0);
    this.#buffer = new Array(actualCapacity);
    this.#start = 0;
    this.#end = 0;
  }

  unshift(value) {
    if (this.#length === this.capacity) {
      this.resize(this.capacity * 2);
    }

    this.#start = (this.#start - 1 + this.capacity) % this.capacity;
    this.#buffer[this.#start] = value;
    this.#length++;

    return this.length;
  }

  push(value) {
    if (this.#length === this.capacity) {
      this.resize(this.capacity * 2);
    }

    this.#buffer[this.#end] = value;
    this.#end = (this.#end + 1) % this.capacity;
    this.#length++;

    return this.length;
  }
}

```

```

shift() {
  if (this.#length === 0) {
    return undefined;
  }

  const value = this.#buffer[this.#start];
  this.#buffer[this.#start] = undefined;
  this.#start = (this.#start + 1) % this.capacity;
  this.#length--;

  return value;
}

pop() {
  if (this.#length === 0) {
    return undefined;
  }

  this.#end = (this.#end - 1 + this.capacity) % this.capacity;
  const value = this.#buffer[this.#end];
  this.#buffer[this.#end] = undefined;
  this.#length--;

  return value;
}

resize(newCapacity) {
  const newBuffer = new Array(newCapacity);

  for (let i = 0; i < this.#length; i++) {
    newBuffer[i] = this.#buffer[(this.#start + i) % this.capacity];
  }

  this.#buffer = newBuffer;
  this.#start = 0;
  this.#end = this.#length;
}

```


second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

БЕЗ ПАНИКИ

- * Запутаться в математике действительно очень просто – **попытайтесь понять паттерн с движением по кругу**
- * Теперь **память буфера всегда используется полностью**
- * Но «**кучность**» **данных уменьшилась**, что скажется на эффективности кэша
- * Является ли это проблемой или нет – **зависит от ситуации**
- * Для расширения размера дека **также используется реаллокация**
- * А с математикой вам поможет ИИ если что 😊

А ЕСЛИ
НЕ ДЕЛАТЬ
РАСШИРЕНИЕ?



ТУТ ДВА ВАРИАНТА

- * Либо после полного заполнения буфера мы **не можем добавлять новые элементы**, пока не появятся свободные места
- * Либо **пишем поверх самых старых данных** продолжая двигаться по кругу
- * Таким образом мы можем сразу выделить необходимую память под структуру и **не делать лишние реаллокации**
- * Бывают ситуации, когда из-за ограничений системы **использовать динамическую память вообще нельзя**

А МОЖНО
ИЗБАВИТЬСЯ
ОТ РЕАЛЛОКАЦИЙ?



ДА, НО ПРИДЕТСЯ ИЗБАВИТЬСЯ ОТ МАССИВА

- * Проблема реаллокации исходит из **фундаментального устройства памяти и массивов**
- * Но вместо хранения непрерывной области памяти, мы можем хранить структуру:
одно значение + указатель на следующую такую структуру в памяти
- * Чтобы понимать, что значений больше нет, договоримся, что
указатель из одних нулей – это `null` и ходить по такому указателю не надо

ЗНАКОМЬТЕСЬ, СВЯЗНЫЙ СПИСОК



```
struct LinkedList<T> {  
    value: T;  
    next: &LinkedList<T>;  
}
```



```
struct LinkedList<T> {  
    value: T;  
    next: &LinkedList<T>;  
}
```



```
struct LinkedList<T> {  
    value: T;  
    next: &LinkedList<T>;  
}
```



```
const linkedList = {  
  value: 42,  
  next: {  
    value: 0,  
    next: {  
      value: 100500,  
      next: null  
    }  
  }  
};  
  
function forEach(head, cb) {  
  while (head !== null) {  
    cb(head.value);  
    head = head.next;  
  }  
}  
  
forEach(linkedList, console.log);
```

В ЧЕМ ТУТ ПРОФИТ

- * Нам не нужно копировать память для расширения – достаточно просто где-то в памяти записать новый узел и сохранить на него ссылку в **next**
- * Чтение первого элемента списка, добавление и удаление в начало – $O(1)$
- * А до остальных нужно сначала дойти, поэтому – $O(n)$
- * Уже хватит для организации стека

```
const linkedList = {
  value: 42,
  next: null,
};

function push(value, linkedList) {
  const next = {...linkedList};
  linkedList.value = value;
  linkedList.next = next;
}

function pop(linkedList) {
  const { value, next } = linkedList;
  linkedList.value = next?.value;
  linkedList.next = next?.next;
  return value;
}

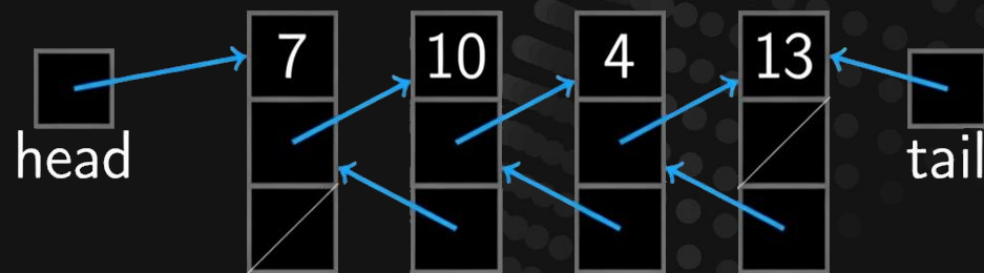
push(100500, linkedList);
push(10, linkedList);

console.log(pop(linkedList)); // 10
console.log(pop(linkedList)); // 100500
console.log(pop(linkedList)); // 42
console.log(pop(linkedList)); // undefined
```


РЕФЛЕКСИРУЕМ

- * Наш стек работает с константной сложностью и не страдает из-за реаллокаций
- * Таким же образом можно реализовать и очередь, но придется сделать некоторые доработки
- * А именно – хранить адреса первого и последнего узлов
- * Такой вид связного списка называется двусторонним
- * И хранить в узлах списка ссылку на следующий и предыдущий узел
- * Такой вид связного списка называется двусвязным или двунаправленным

ДВУСТОРОННИЙ ДВУСВЯЗНЫЙ СПИСОК



```
class LinkedList {
  first = null;
  last = null;

  push(value) {
    const node = { value, next: null, prev: this.last };

    if (this.first == null) {
      this.first = node;
      this.last = node;
      return;
    }

    if (this.last != null) {
      this.last.next = node;
    }

    this.last = node;
  }

  shift() {
    const { first } = this;

    if (first == null) { return undefined; }

    const { value, next } = first;

    if (next != null) {
      next.prev = null;
    }

    this.first = next;
    return value;
  }
}
```

```
const linkedList = new LinkedList();

linkedList.push(42);
linkedList.push(100500);
linkedList.push(10);

console.log(linkedList.shift()); // 42
console.log(linkedList.shift()); // 100500
console.log(linkedList.shift()); // 10
console.log(linkedList.shift()); // undefined
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ВСЕ НЕ ТАК СТРАШНО

- * Двусторонний двусвязный список может быть одной из реализаций для очереди или дека
- * Обладает константной сложностью для всех своих операций и не требует реаллокаций памяти
- * Выглядит как идеальное решение...

ЛОЖКА ДЕГТЯ

- * Нам нужно **хранить не только само значение**, но и указатель на следующий узел
- * Для **двусвязного списка** уже два указателя на узел
- * Но главное – данные связного списка разбросаны по памяти, а значит **это плохо для кеширования**

СРАВНИМ – РАЗНИЦА В 2 РАЗА!

```
const SIZE = 1e7;

for (let i = SIZE - 1; i >= 0; i--) {
  list = { value: i, next: list };
}

function traverseLinkedList(head) {
  let node = head;
  let sum = 0;

  while (node !== null) {
    sum += node.value;
    node = node.next;
  }

  return sum;
}
```

```
const SIZE = 1e7;

const array = Array.from({ length: SIZE }, (_, i) => i);

let list = null;

function traverseArray(arr) {
  let sum = 0;

  for (let i = 0; i < arr.length; i++) {
    sum += arr[i];
  }

  return sum;
}
```

РЕФЛЕКСИРУЕМ

- * Связный список не нуждается в копировании памяти при расширении, но **куда хуже использует кэш-процессора**
- * Это главная причина, почему на практике **связный список «в чистом виде» используется редко**
- * Но как часть другой структуры данных – уже гораздо чаще
- * Например, можно сделать **связный список массивов...**

СРАВНИМ – РАЗНИЦЫ ПОЧТИ НЕТ!

```
const SIZE = 2 ** 18;
const BLOCK_SIZE = 64;
const NUM_BLOCKS = Math.ceil(SIZE / BLOCK_SIZE);

let listArray = null;

for (let i = NUM_BLOCKS - 1; i >= 0; i--) {
  const block = new Array(BLOCK_SIZE);
  const startIdx = i * BLOCK_SIZE;

  for (let j = 0; j < BLOCK_SIZE; j++) {
    const idx = startIdx + j;
    block[j] = idx < SIZE ? idx : undefined;
  }

  listArray = { value: block, next: listArray };
}
```

```
const SIZE = 2 ** 18;

const array = Array.from({ length: SIZE }, (_, i) => i);

let list = null;

function traverseArray(arr) {
  let sum = 0;

  for (let i = 0; i < arr.length; i++) {
    sum += arr[i];
  }

  return sum;
}
```

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

СТОЯНА, СТОЯНААА!

ЭТО ЖЕ КРУТО!

- * Программирование – это **балансирование компромиссами**
- * Сделав связный список массивов мы **улучшили работу с кэшем**, так и снизили общее потребление памяти (меньше указателей)
- * Размер блока нужно выбирать таким образом, **чтобы все его элементы полностью заполняли кэш-линии**
- * При этом он **не должен быть слишком большим или маленьким** – от этого зависит потребление памяти и количество аллокаций

МОЖНО СДЕЛАТЬ ДЕК
НА СВЯЗНОМ СПИСКЕ
МАССИВОВ?



ВПОЛНЕ

- ✱ Реализация будет сложнее, но **не нужно копировать память при расширении**
- ✱ Как и **не так сильно зависим от задания стартовой емкости**
- ✱ Такой подход часто используется на практике

РЕЗЮМИРУЕМ

- ✱ Если размер фиксирован – массив это самая лучшая структура
- ✱ А вот **дальше смотрим и думаем**, что нам важнее – производительность в среднем или потребление памяти

А ПОЧЕМУ БЫ
И **ВЕКТОР** ТАК
НЕ СДЕЛАТЬ?



МОЖНО

- * Но тогда мы теряем константный доступ к элементу по индексу –
нам же **сначала нужно перейти к нужному узлу списка**
- * Можно придумать различные ухищрения, как ускорить этот процесс –
так и делают различные структуры данных (например, B-дерево)
- * Но сделать быстрее и проще, чем обычный массив все равно не выйдет,
а вектор в основном и используется как обычный массив
- * Поэтому **реализация с копированием в среднем самая лучшая**,
а для других кейсов надо использовать другие структуры

second monitor

\$0.00 (0%)

50.00

224 DAYS TO 20

\$160.00

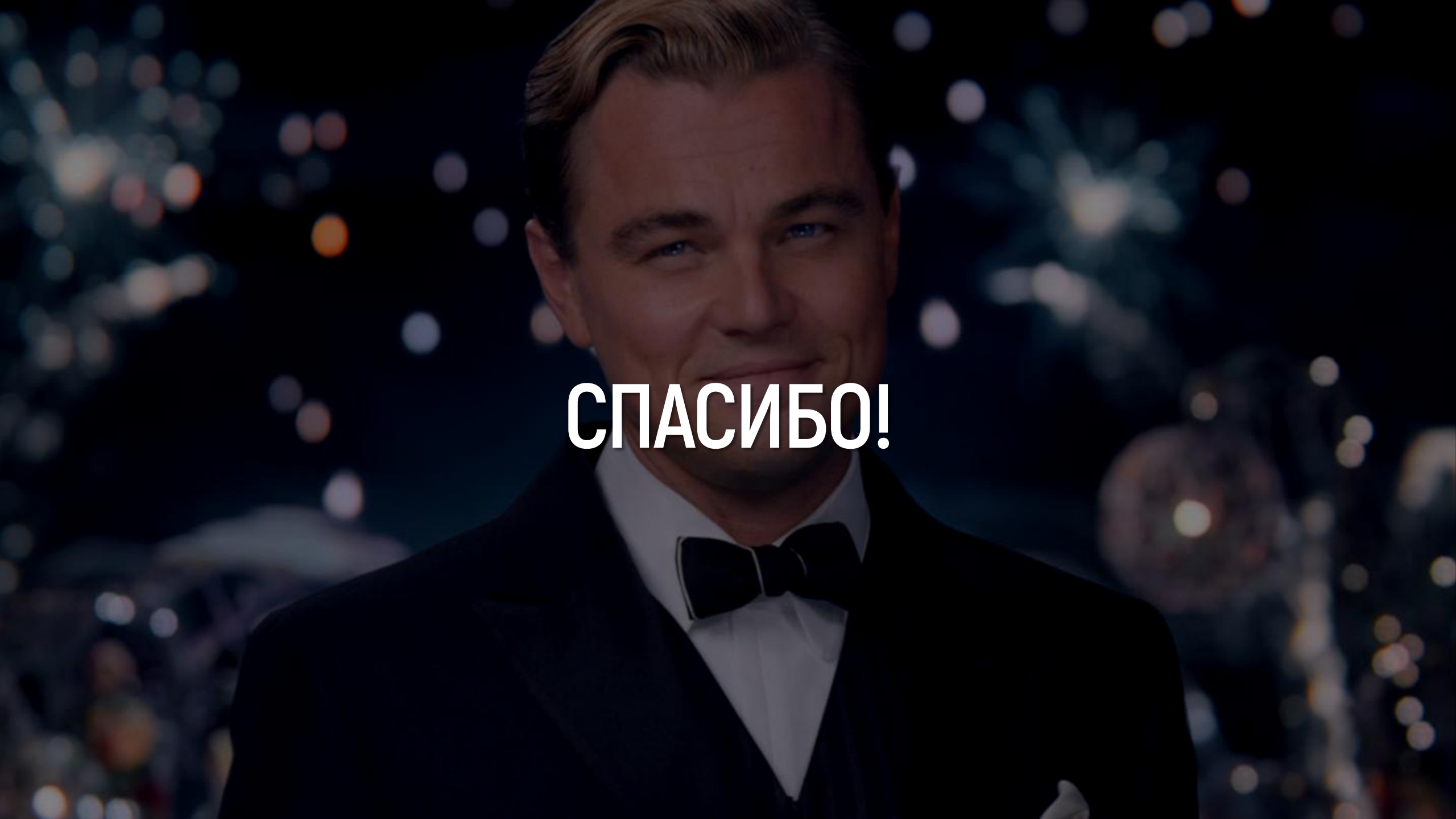
СТОЯНА, СТОЯНААА!

ВЫДЫХАЕМ

- * Главное поймите – **идеальной структуры данных нет**
- * Есть только **наиболее подходящие под конкретные задачи**
- * Задача инженера – проанализировать **какая структура лучше «здесь и сейчас»**

ПОДВЕДЕМ ИТОГИ

- * Абстрактные структуры данных – **просто вводят контракты на API**, но не говорят как именно его реализовать
- * Стек и очередь – **две важнейших абстрактных структуры данных**
- * Дек – это просто **двусторонняя очередь**
- * **Реализаций этих структур существует много** – идеального нет
- * Связные список **позволяет динамически «растить» память** без необходимости копирования, но хуже работает с кэшем
- * Связный список очень хорош, как **часть более сложной структуры**

A close-up portrait of Leonardo DiCaprio, smiling slightly, wearing a dark tuxedo with a white shirt and a dark bow tie. The background is dark with out-of-focus bokeh lights in various colors (white, yellow, orange, blue), suggesting a night-time event or fireworks.

СПАСИБО!